CALCULATOR WEB APP

📌 PROJECT TITLE

Web-Based Calculator using Flask

🎯 OBJECTIVE

To develop a simple web application that performs basic arithmetic operations using Flask framework and handles user input through HTML forms.

🛠TECHNOLOGIES USED

- Python
- Flask
- HTML

⚙KEY CONCEPTS USED

- routing (@app.route)
- GET & POST methods
- Form handling
- Template rendering

📁 PROJECT STRUCTURE

```
calculator_app/
│
├── app.py
└── templates/
    └── index.html
```

📄 SOURCE CODE (app.py)

```
from flask import Flask, render_template, request

app = Flask(__name__)

@app.route("/", methods=["GET", "POST"])
def calculator():
```

```python
    result = ""

    if request.method == "POST":
        num1 = float(request.form["num1"])
        num2 = float(request.form["num2"])
        operation = request.form["operation"]

        if operation == "add":
            result = num1 + num2
        elif operation == "sub":
            result = num1 - num2
        elif operation == "mul":
            result = num1 * num2
        elif operation == "div":
            result = num1 / num2

    return render_template("index.html", result=result)

if __name__ == "__main__":
    app.run(debug=True)
```

🎯 HTML CODE (templates/index.html)

```html
<!DOCTYPE html>
<html>
<head>
<title>Flask Calculator</title>
</head>

<body>

<h2>Simple Flask Calculator</h2>

<form method="POST">

Number 1:
<input type="number" name="num1" required><br><br>

Number 2:
<input type="number" name="num2" required><br><br>

<select name="operation">
<option value="add">Add</option>
<option value="sub">Subtract</option>
<option value="mul">Multiply</option>
<option value="div">Divide</option>
</select>
```

```
<br><br>

<button type="submit">Calculate</button>

</form>

<h3>Result: {{ result }}</h3>

</body>
</html>
```

▸ EXECUTION

python app.py

🔄 WORKING FLOW

- User enters numbers
- Selects operation
- Form sends data to Flask
- Flask processes logic
- Result displayed on webpage

💡 PROFESSIONAL ENHANCEMENTS

- Add **error handling** (invalid input)
- Use **Bootstrap UI**
- Add **history of calculations**
- Convert to **REST API**

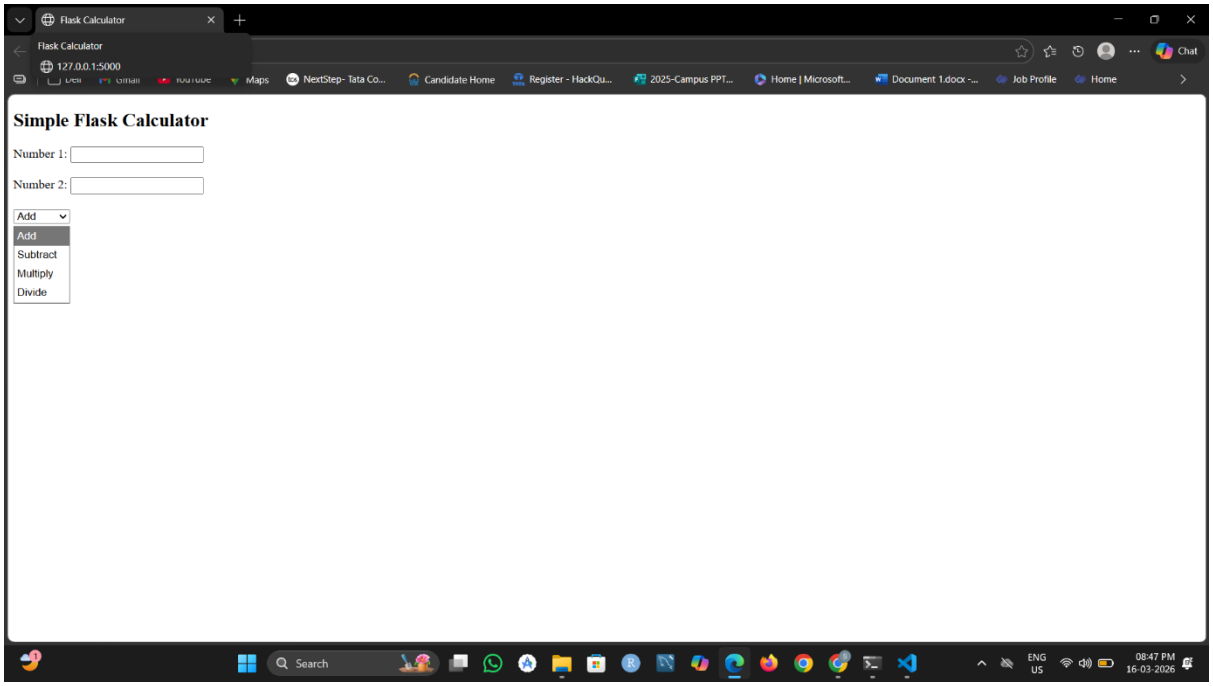
👉This project demonstrates form handling and routing in Flask. It processes user input and performs arithmetic operations dynamically on the server side

✅ OUTCOMES

- understood the basics of Flask framework
- learned how to handle user input using HTML forms
- Implemented arithmetic operations using Python logic

- Gained knowledge of GET and POST request methods

- developed a simple web-based application



TO-DO LIST WEB APP

📌 PROJECT TITLE

Task Management System (To-Do List) using Flask

🎯 OBJECTIVE

To develop a web-based application that allows users to manage daily tasks by adding, viewing, and deleting tasks using Flask.

🛠 TECHNOLOGIES USED

- Python
- Flask
- HTML / CSS

⚙ KEY CONCEPTS USED

- CRUD operations (Create, Read, Delete)
- Flask routing
- Template rendering
- Form handling

□ PROJECT STRUCTURE

```
todo_app/
│
├── app.py
└── templates/
    └── index.html
```

□ SOURCE CODE (app.py)

```
from flask import Flask, render_template, request, redirect

app = Flask(__name__)

tasks = []   # store tasks in list
```

```python
@app.route("/", methods=["GET", "POST"])
def index():
    if request.method == "POST":
        task = request.form["task"]
        if task != "":
            tasks.append(task)
        return redirect("/")
    return render_template("index.html", tasks=tasks)

@app.route("/delete/<int:id>")
def delete(id):
    tasks.pop(id)
    return redirect("/")

if __name__ == "__main__":
    app.run(debug=True)
```

🎨 HTML CODE (templates/index.html)

```html
<!DOCTYPE html>
<html>
<head>
    <title>To-Do App</title>
</head>
<body>

    <h2>📝 My To-Do List</h2>

    <!-- Add Task Form -->
    <form method="POST">
        <input type="text" name="task" placeholder="Enter task">
        <button type="submit">Add</button>
    </form>

    <!-- Display Tasks -->
    <ul>
        {% for task in tasks %}
            <li>
                {{ task }}
                <a href="/delete/{{ loop.index0 }}">❌</a>
            </li>
        {% endfor %}
    </ul>

</body>
</html>
```

⍰ EXECUTION

python app.py

🔄 WORKING FLOW

- User enters a task
- Task is sent to Flask
- Flask stores task in list
- Tasks are displayed on page
- User can delete tasks

IMPORTANT:

□ ✅ Mark task as completed

□ ✅ Use **SQLite database** (very important)

□ ✅ Add timestamps (date & time)

□ ✅ Use Bootstrap UI

□ ✅ Add edit/update task feature

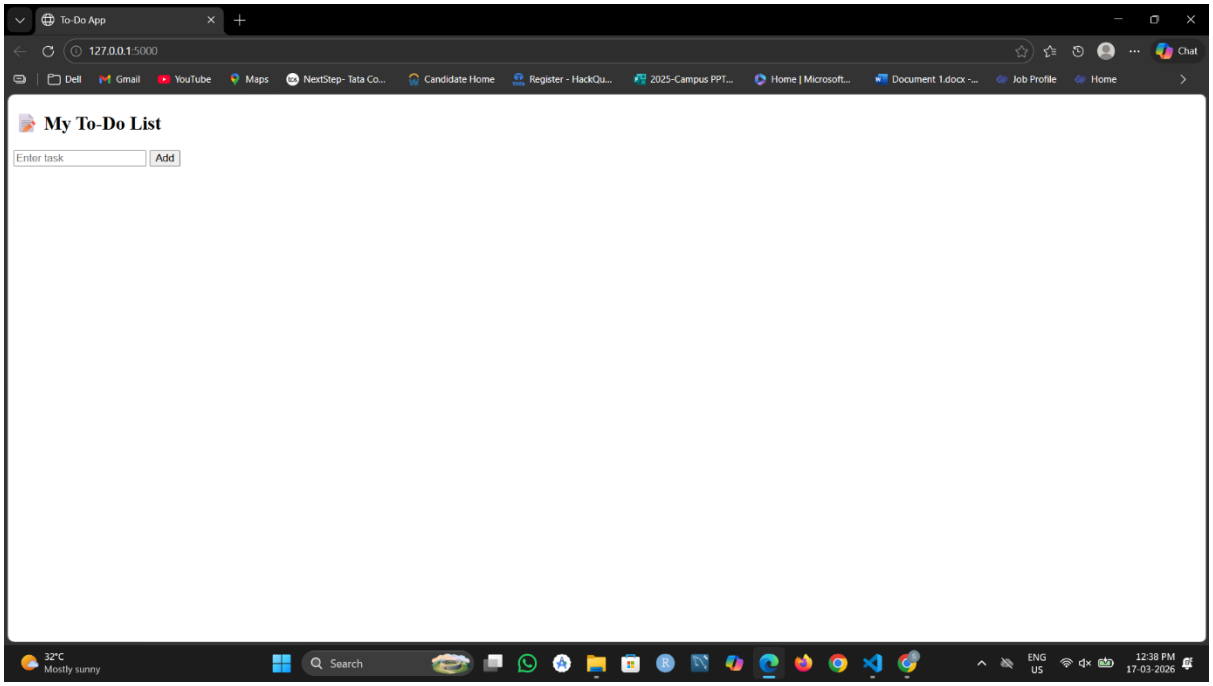
📄 LEARNING OUTCOMES

- developed a task management web application using Flask
- Implemented Create, Read, and Delete operations
- Managed dynamic data using server-side logic
- Demonstrated understanding of web application flow

👉 I developed a To-Do List application using Flask that performs CRUD operations and manages tasks dynamically on the server side

🏁 OUTCOMES

- Understand CRUD operations clearly
- Learn real-world app structure
- Prepare for advanced projects like Login System


To-Do App
127.0.0.1:5000
My To-Do List
Enter task
Add

LOGIN SYSTEM APP

📌 PROJECT TITLE

User Authentication System using Flask

🎯 OBJECTIVE

To develop a secure web application that authenticates users using login credentials and manages sessions using Flask.

🛠 TECHNOLOGIES USED

- Python
- Flask
- HTML
- (Optional: SQLite for database)

⚙ KEY CONCEPTS USED

- Authentication (Login/Logout)
- Session management
- Routing & redirection
- Form handling
- Security basics

□ PROJECT STRUCTURE

```
login_app/
│
├── app.py
└── templates/
    ├── login.html
    └── dashboard.html
```

□ SOURCE CODE (app.py)

```python
from flask import Flask, render_template, request, redirect, session, url_for

app = Flask(__name__)
app.secret_key = "secret123"   # required for session

# Dummy user data
USERNAME = "admin"
PASSWORD = "1234"

@app.route("/", methods=["GET", "POST"])
def login():
    error = ""
    if request.method == "POST":
        username = request.form["username"]
        password = request.form["password"]

        if username == USERNAME and password == PASSWORD:
            session["user"] = username
            return redirect(url_for("dashboard"))
        else:
            error = "Invalid Credentials ❌"

    return render_template("login.html", error=error)

@app.route("/dashboard")
def dashboard():
    if "user" in session:
        return render_template("dashboard.html", user=session["user"])
    return redirect(url_for("login"))

@app.route("/logout")
def logout():
    session.pop("user", None)
    return redirect(url_segment("login"))

if __name__ == "__main__":
    app.run(debug=True)
```

🎨 HTML CODE (login.html)

```html
<!DOCTYPE html>
<html>
<head>
    <title>Login</title>
</head>
<body>
```

```
    <h2>Login Page</h2>

    <form method="POST">
        Username: <input type="text" name="username"><br><br>
        Password: <input type="password" name="password"><br><br>
        <button type="submit">Login</button>
    </form>

    <p style="color:red;">{{ error }}</p>
</body>
</html>
```

HTML CODE (dashboard.html)

```
<!DOCTYPE html>
<html>
<head>
    <title>Dashboard</title>
</head>
<body>
    <h2>Welcome {{ user }} 🎉</h2>
    <a href="/logout">Logout</a>
</body>
</html>
```

▯ EXECUTION

python app.py

🔄 WORKING FLOW

- User enters username & password
- Flask validates credentials
- if correct → session created
- User redirected to dashboard
- Logout removes session

🎓 LEARNING OUTCOMES

- developed a user authentication system using Flask
- Implemented session-based access control
- Handled user credentials and validation

• Demonstrated understanding of secure web application flow

IMPORTANT:

- ✅ **SQLite database** (store users)
- ✅ **Password hashing** (werkzeug.security)
- ✅ **Registration page**
- ✅ **Flash messages**
- ✅ **Bootstrap UI**
- ✅ **Login validation (email format, strong password)**

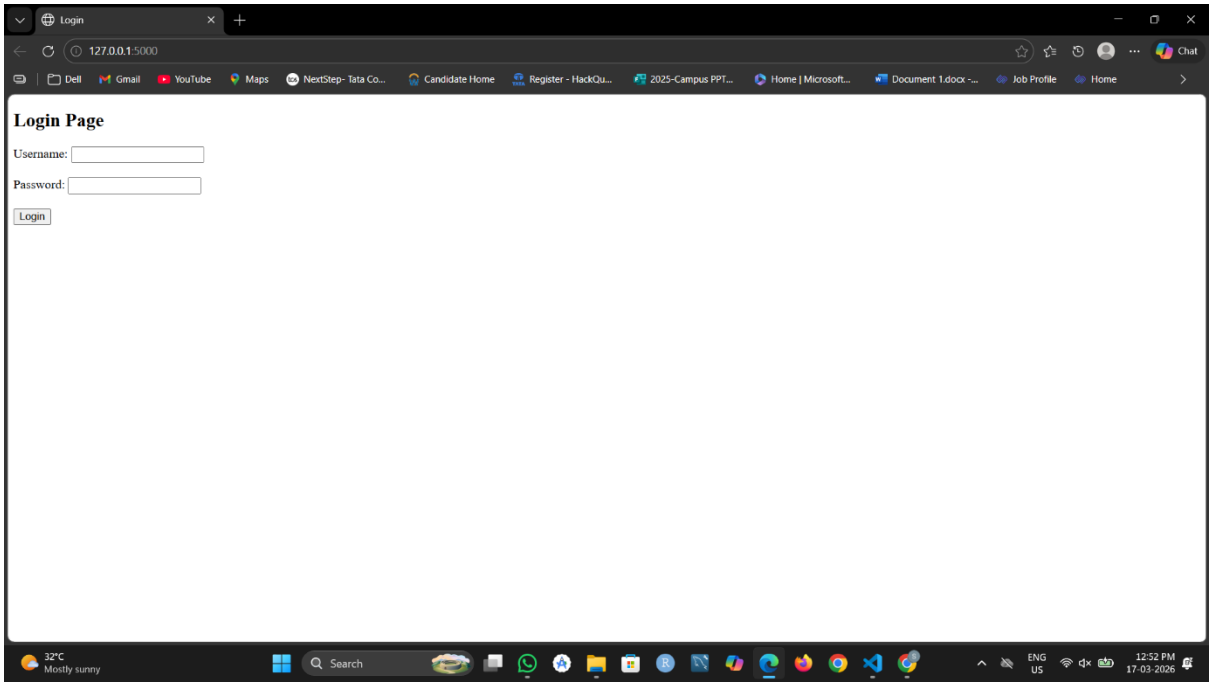
  ☞ I developed a login authentication system using Flask with session management. It validates user credentials and restricts unauthorized access. I can extend it with database integration and password encryption for production use

🏁 OUTCOMES

• Understand real-world authentication system

• Learn session handling (very important in interviews)

• Ready for advanced projects



🏆 FINAL NOTE POINTS

☞

- These projects demonstrate the basics of web development using Flask
- They cover essential concepts like routing, form handling, CRUD operations, and authentication
- They help in building real-world applications with user interaction